Wrocław
University
of Science
and Technology

Identity and Access Management

Cloud Programming (W04IST-SI0826G)

Wojciech Thomas, PhD

Department of Applied Informatics
Faculty of Information and Communication Technology

Spring 2026

This week

After this class, you should be able to:

- Explain the purpose of AWS IAM and why it is essential for securing S3, RDS, and DynamoDB
- Distinguish between IAM Users, Groups, Roles, and Policies and choose the right construct
- Interpret an IAM policy JSON document and predict the permissions it grants
- Apply the principle of least privilege when designing IAM configurations
- Explain how AWS STS and IAM Identity Center simplify multi-account access

Why this matters

- A developer is given `AdministratorAccess` to quickly fix a production bug
- Two weeks later a misconfigured script runs with those same credentials
- An entire S3 bucket containing production data is **permanently deleted**
- Backups were not configured — recovery is impossible
- **One scoped IAM policy would have prevented the entire incident**

Wrocław University of Science and Technology

Agenda

1 IAM Core Concepts

2 Roles, STS, and Identity Center

3 IAM Best Practices

Quick recall

Without looking at your notes:

- Name two PaaS services from last week that store persistent data
- When your application reads from an S3 bucket, how does AWS decide if it is allowed?
- What does “private by default” mean for an S3 bucket?

- S3 and RDS (or DynamoDB)
- AWS evaluates the identity and permissions attached to the request — via IAM
- No public access unless explicitly granted; all requests are denied by default

Quick recall

Without looking at your notes:

- Name two PaaS services from last week that store persistent data
- When your application reads from an S3 bucket, how does AWS decide if it is allowed?
- What does “private by default” mean for an S3 bucket?

- S3 and RDS (or DynamoDB)
- AWS evaluates the identity and permissions attached to the request — via IAM
- No public access unless explicitly granted; all requests are denied by default

What is AWS IAM?

- Controls who can do what on which AWS resources
- Applies to every AWS service — S3, RDS, DynamoDB, Lambda all use IAM
- Core principle: **deny by default** — access must be explicitly granted
- Free service; no additional cost

Identity and Access Management (IAM)

AWS service for managing **authentication** (who you are) and **authorization** (what you may do) across all AWS resources.

Root Account

- Automatically created when an AWS account is opened
- Has unrestricted access to every AWS service and resource
- Cannot be limited by any IAM policy — even `Deny` policies cannot restrict root
- **Must never be used for day-to-day work**
- Best practice: enable MFA on root, then store credentials offline and do not use them

Recall the opening scenario: `AdministratorAccess` is root-equivalent in practice. Root is worse — it cannot be restricted at all.

IAM Users

- Represents a single person or application interacting with AWS
- Associated with exactly one AWS account
- Starts with **zero permissions** — must be explicitly granted
- Two credential types:
  - **Password** — AWS Management Console (web UI access)
  - **Access key** — AWS CLI and SDK (programmatic access)
- One-to-one mapping enables individual accountability and auditing

IAM Groups

- A named collection of IAM users sharing the same permissions
- Policies are attached to the group, not to individual users
- Adding a user to a group instantly grants all group permissions
- A user can belong to **multiple groups**
- Groups cannot contain other groups
- Examples: `Developers`, `DBAdmins`, `ReadOnly`, `Billing`

Before we continue — predict

You create a brand-new IAM user and add them to a group called `Developers`.

The group has **no policies attached** yet.

The user tries to list all S3 buckets. What happens?

- Access is denied — groups without attached policies grant nothing
- IAM never assumes, inherits, or defaults to any access
- Every permission must be explicitly granted

Before we continue — predict

You create a brand-new IAM user and add them to a group called `Developers`.

The group has **no policies attached** yet.

The user tries to list all S3 buckets. What happens?

- Access is **denied** — groups without attached policies grant nothing
- IAM never assumes, inherits, or defaults to any access
- Every permission must be explicitly granted

IAM Policies

- JSON documents that define permissions
- Attached to users, groups, or roles
- Each **statement** contains:
  - `Effect` — `Allow` or `Deny`
  - `Action` — API call(s) e.g. `s3:GetObject`, `ec2:*`
  - `Resource` — ARN identifying the target resource
  - `Condition` *(optional)* — IP range, MFA required, time window

```
{
  "Effect": "Allow",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::my-bucket/*"
}
```

Policy Types

- Identity-based — attached to a user, group, or role (most common)
- **Resource-based** — attached directly to a resource (e.g., S3 bucket policy)
- **Permissions boundaries** — cap the *maximum* permissions an identity may ever receive
- **Service Control Policies (SCPs)** — org-level guardrails applied across all accounts

For most individual use cases, identity-based policies are sufficient. SCPs and boundaries become important when managing organizations at scale.

Wrocław University of Science and Technology

Agenda

1 IAM Core Concepts

2 Roles, STS, and Identity Center

3 IAM Best Practices

IAM Roles

- An identity with permissions — not tied to a specific person
- No long-term credentials: no password, no permanent access key
- Any **trusted entity** can *assume* a role and receive temporary credentials
- Common use cases:
  - EC2 instance reading objects from S3
  - Lambda function writing records to DynamoDB
  - Cross-account access between two AWS accounts
  - Federated login (corporate Active Directory -> AWS console)

Users vs. Roles

	IAM User	IAM Role
Credentials	Long-term (key / password)	Temporary (STS token, expires)
Tied to	Specific person or app	Any trusted entity that assumes it
Use when	Human console / CLI access	Services, automation, cross-account
Revocation	Must delete or rotate manually	Token expires automatically

Embed a role on an EC2 instance instead of hardcoding access keys. Keys can be leaked to git; role tokens expire and rotate silently.

Admin challenge

*Your Java Spring Boot backend on EC2 needs to read secrets from **AWS Secrets Manager** and write logs to an **S3 bucket**.*
A colleague says: "Just create an IAM user, generate an access key, and paste it into the app config."
*What are the **three problems** with this approach? What should you do instead?*

1. Access keys in config leak via git commits, logs, or Docker image layers
2. Long-term credentials persist until manually rotated — a leak is permanent until noticed
3. Hard to enforce least privilege per application as the codebase grows

Solution: Create an IAM Role scoped to Secrets Manager and the specific S3 bucket; attach it as an EC2 instance profile

Wrocław University of Science and Technology

Admin challenge

*Your Java Spring Boot backend on EC2 needs to read secrets from **AWS Secrets Manager** and write logs to an **S3 bucket**.*
A colleague says: "Just create an IAM user, generate an access key, and paste it into the app config."
*What are the **three problems** with this approach? What should you do instead?*

1. Access keys in config leak via git commits, logs, or Docker image layers
2. Long-term credentials persist until manually rotated — a leak is permanent until noticed
3. Hard to enforce least privilege per application as the codebase grows

Solution: Create an IAM Role scoped to Secrets Manager and the specific S3 bucket; attach it as an EC2 instance profile

AWS Security Token Service (STS)

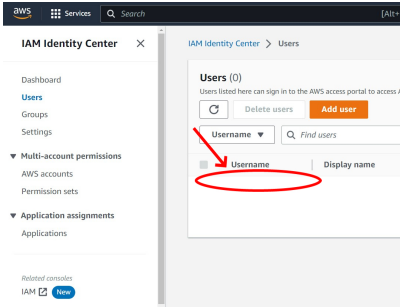
- Issues temporary security credentials when a role is assumed
- Returned credentials include:
  - Access Key ID + Secret Access Key
  - **Session Token** (mandatory alongside the key pair)
- Configurable validity: 15 minutes to 12 hours
- AWS SDKs call STS automatically when an EC2 instance role is configured
- Key API call: `sts:AssumeRole` — the mechanism behind all role assumption

How it works

`AssumeRole` IAM validates trust policy STS returns timed credentials SDK uses them credentials expire and rotate

AWS IAM Identity Center

- Centralized access management across multiple AWS accounts and applications
- Successor to the legacy AWS Single Sign-On (SSO) service
- Key capabilities:
  - **Single Sign-On** — one login for all accounts and SaaS apps
  - Integration with external IdPs: Microsoft AD, Okta, Google Workspace
  - **Permission sets** mapped to accounts (built on IAM Roles internally)
  - Unified audit trail across all accounts in one place



IAM vs. IAM Identity Center

	AWS IAM	AWS IAM Identity Center
Scope	Single AWS account	Multiple accounts + SaaS apps
Users	IAM users in the account	Users from IdP or built-in directory
Authentication	AWS password / access key	SSO (SAML 2.0 / OIDC)
Best for	Single account, automation	Human workforce at scale

*AWS recommends IAM Identity Center for all **human** access in organizations. Use IAM Roles directly for **workload** (machine) access.*

Agenda

1 IAM Core Concepts

2 Roles, STS, and Identity Center

3 IAM Best Practices

Wrocław University of Science and Technology

Discuss with your neighbour (2 min)

Read this policy carefully:

```
{
  "Effect": "Allow",
  "Action": "*",
  "Resource": "*"
}
```

- What can this identity do?
- What can it *not* do?
- Is this a well-designed policy? Why or why not?

IAM Best Practices

- **Lock the root account** — enable MFA; never use it for daily operations
- **Grant least privilege** — start with minimum permissions; expand only as required
- **Use roles for workloads** — never embed long-term access keys in application code
- **Enable MFA** for all human IAM users with console access
- **Use IAM Identity Center** for human workforce access across accounts
- **Review and remove** unused users, roles, keys, and policies regularly
- **Audit with IAM Access Analyzer** — identifies unintended external and cross-account access

IAM in Context

Every PaaS service from Lecture 6 enforces access through IAM:

- **S3** — bucket policies + IAM policies control every object read and write
- **RDS** — supports IAM database authentication as an alternative to passwords
- **DynamoDB** — IAM policies restrict access per table, conditionally per item
- **Lambda** — every function has an *execution role* defining what it can call
- **Elastic Beanstalk** — uses an EC2 instance profile role to reach S3 and CloudWatch

Without a correctly configured IAM policy, none of your application code reaches these services.

Before you go

On a piece of paper (or in your notes), write:

1. One thing from today you are **confident** about
2. One thing you are **still unsure** about — bring it to the next class

Self-check questions:

- Can you name the four IAM building blocks and explain what each does?
- Can you explain *why* you would use a Role instead of a User for an EC2 application?
- Can you read a policy JSON and state what it allows and to which resource?

Wrocław University of Science and Technology

References

[1] Amazon Web Services, “AWS identity and access management user guide,” 2026. https://docs.aws.amazon.com/IAM/latest/UserGuide/ (accessed May 04, 2026).

[2] Amazon Web Services, “AWS security token service API reference,” 2026. https://docs.aws.amazon.com/STS/latest/APIReference/ (accessed May 04, 2026).

[3] Amazon Web Services, “AWS IAM identity center user guide,” 2026. https://docs.aws.amazon.com/singlesignon/latest/userguide/ (accessed May 04, 2026).

[4] Amazon Web Services, “AWS well-architected framework — security pillar,” 2026. https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html (accessed May 04, 2026).

Wrocław University of Science and Technology

Thank you for your attention

- See you next week